

SQL 注入漏洞报告 (Lab5: UNION 攻击从其他表获取数据)

1. 漏洞名称

SQL Injection (SQL 注入) — UNION 联合查询跨表数据窃取

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站商品分类查询功能存在严重的 SQL 注入漏洞。结合前序实验 (已确定查询包含 3 列, 且第 2 列可用于显示文本), 攻击者可以通过 UNION 关键字构造恶意 SQL 语句, 从数据库的 users 表中直接读取所有用户的用户名和密码, 并回显在网页上, 最终导致攻击者使用管理员账号成功登录后台。

4. 漏洞位置

请求方式: GET /filter?category=...

漏洞参数: category

5. 漏洞复现过程

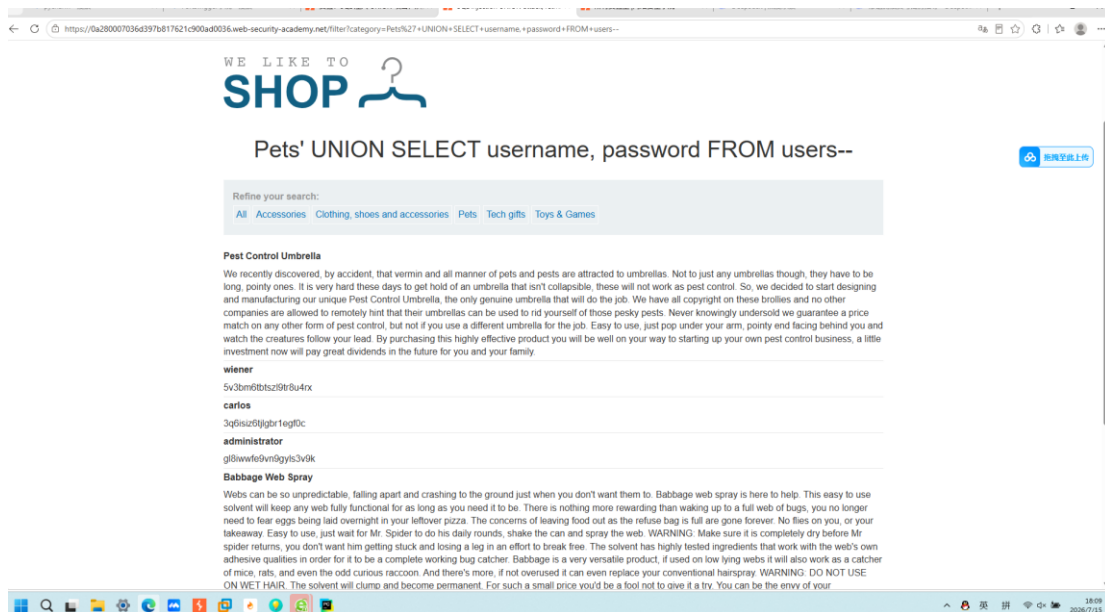
第一步: 构造跨表查询 Payload

基于前期探测结果, 攻击者发现原始查询有 3 列, 且第 2 列是文本列 (适合放用户名, 密码也是字符串), 第 3 列是其他类型 (设为 null)。

构造最终攻击 Payload:

text

Pets' UNION SELECT username, password FROM users—



截图 1：成功读取数据库用户表（关键截图）

说明：执行 Payload 后，页面列表下方出现了原数据库中的账号密码信息。这证明了攻击者已经成功从 users 表中提取出了敏感身份凭证。

第二步：利用窃取到的凭证登录后台

获取到管理员（administrator）的明文密码（如 gl8www9vn9yis3v9k）后，访问 /login 页面，输入管理员账号及该密码。



截图 2：成功登录管理员后台

说明：使用 SQL 注入窃取到的管理员账号密码成功通过身份验证，直接接管了网站的管理员权限。

第三步：漏洞原理

原 SQL:

sql

```
SELECT * FROM products WHERE category='Pets'
```

经过攻击者构造的 Payload 拼接后，后端实际执行的 SQL 变成了：

sql

```
SELECT * FROM products WHERE category='Pets' UNION SELECT username, password  
FROM users --'
```

- UNION：将后续的 SELECT 结果拼接到第一个查询结果后面。
- FROM users：直接读取数据库中保存用户的表。
- 原本只能显示“商品”的地方，变成了显示“用户名”和“密码”。

6. 漏洞影响

这是最典型的**敏感数据泄露（数据外带）**攻击：

- 攻击者可直接获取数据库内所有用户的明文密码（如果未加密）。
- 导致攻击者任意接管高权限账号（管理员）。
- 导致用户数据泄露，严重破坏系统机密性。

7. 修复建议

- ① **使用参数化查询（预编译）**：这是防范 SQL 注入最根本的手段，彻底切断恶意 SQL 语句的拼接。
- ② **密码禁止明文存储**：数据库中的密码必须经过哈希加密（如 bcrypt 等）存储，即便数据库被攻破，攻击者也无法直接拿到明文密码登录。
- ③ **最小权限原则**：常规查询用的数据库账号，**绝对不要赋予**读取其他敏感表（如 users）的权限。

8. 工具

- Google Chrome / Firefox 浏览器（修改 URL 参数）
- PortSwigger Web Security Academy（靶场平台）